

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 3: Comunicación y Servicios

Introducción al Módulo

En el desarrollo de aplicaciones robustas con Angular, la gestión de la información y la interacción entre componentes representan pilares fundamentales para garantizar la escalabilidad y mantenibilidad del software. Este módulo profundiza en los mecanismos esenciales que permiten acoplar de forma holgada las vistas de nuestra aplicación, centralizar las reglas de negocio en servicios reutilizables y dominar el ciclo de vida de los elementos de la interfaz.

A través de esta guía, analizaremos tanto los enfoques clásicos basados en decoradores y constructores como las metodologías modernas consolidadas en Angular 19, complementándolo con el glosario conceptual (ver ANEXO I), la matriz operativa de ciclo de vida e inyección (ver ANEXO II), las nuevas primitivas de comunicación por señales (ver ANEXO III), la jerarquía de inyectores (ver ANEXO IV), las estrategias de prevención de fugas de memoria (ver ANEXO V) y los fundamentos de TypeScript aplicados a servicios (ver ANEXO VI), proporcionando un marco técnico óptimo para el desarrollo profesional.

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **1 Comunicación entre Componentes (Inputs, Outputs, EventEmitter):** Flujo de datos unidireccional, parametrización descendente y emisión de eventos hacia el componente padre.
- **2 Servicios e Inyección de Dependencias (@Injectable):** Centralización de lógica de negocio, alcance global con el patrón Singleton y optimización de compilación con Tree-Shaking (ver ANEXO I y ANEXO IV).
- **3 Ciclo de Vida de un Componente:** Fases de ejecución gestionadas por el framework, interfaces Lifecycle Hooks e intercepción segura de renderizado (ver ANEXO II y ANEXO V).
- **4 Uso de inject() en lugar de Constructor para Inyección Moderna:** Paradigma funcional en Angular, resolución de dependencias sin constructor y contexto de inyección (ver ANEXO I y ANEXO II).
- **5 Ejercicio Final:** Construcción completa de un gestor de tareas desacoplado coordinando servicios compartidos y componentes autónomos en Angular 19.
- **6 Resumen Ejecutivo del Conocimiento Adquirido:** Síntesis formal de competencias, directrices de arquitectura y patrones de ingeniería asimilados.

Anexos Técnicos de Referencia:

- **ANEXO I (Glosario de Términos Técnicos):** Compendio conceptual con la definición rigurosa de términos citados con (ver ANEXO I).
- **ANEXO II (Matriz del Ciclo de Vida y Estrategias de Inyección):** Tabla de consulta rápida, miniejercicios de intercepción y ejercicio integrador, citado con (ver ANEXO II).
- **ANEXO III (Nuevas Primitivas de Señales en Comunicación):** Primitivas reactivas `input()`, `output()` y `model()` en Angular 19, citado con (ver ANEXO III).
- **ANEXO IV (Ámbitos de Inyección y Jerarquía de Inyectores):** Inyector raíz, proveedores locales de componente y modificadores de resolución, citado con (ver ANEXO IV).
- **ANEXO V (Prevención de Fugas de Memoria en Componentes):** Detección de fugas, operador `takeUntilDestroyed()` y consumo con `AsyncPipe`, citado con (ver ANEXO V).
- **ANEXO VI (Fundamentos de TypeScript para Servicios):** Interfaces de dominio, modificadores de acceso, inmutabilidad y servicios genéricos `<T>`, citado con (ver ANEXO VI).

1 Comunicación entre Componentes (Inputs, Outputs, EventEmitter)

Angular propone una arquitectura basada en componentes jerárquicos. Para que estos componentes permanezcan independientes y reutilizables, la comunicación debe ser estructurada y predecible mediante un flujo de datos unidireccional: datos hacia abajo (Inputs), eventos hacia arriba (Outputs).

1.1 El Decorador @Input()

El decorador `@Input()` permite definir propiedades en un componente hijo que pueden recibir valores desde su componente padre. Esto dota al componente de la capacidad de parametrización en arquitectura autónoma.

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-todo-item',
  standalone: true,
  template: `<p>Tarea: {{ taskName }}</p>`
})
export class TodoItemComponent {
  @Input() taskName: string = '';
}
```

Listing 1: p1-1-todo-item.component.ts

```
<!-- Componente Padre -->
<app-todo-item [taskName]="Aprender Angular"></app-todo-item>
```

Listing 2: p1-1-parent.component.html

1.2 El Decorador @Output() y EventEmitter

Para enviar notificaciones o transferir datos de vuelta al componente padre, el hijo utiliza el decorador @Output() en combinación con una instancia de la clase EventEmitter. El hijo emite un evento y el padre escucha dicho evento mediante el mapeo de eventos en su plantilla HTML.

```
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-todo-item',
  standalone: true,
  template: `<button (click)="notifyParent()">Completar</button>`
})
export class TodoItemComponent {
  @Output() completed = new EventEmitter<void>();

  notifyParent(): void {
    this.completed.emit();
  }
}
```

Listing 3: p1-2-output-item.component.ts

```
<!-- Componente Padre -->
<app-todo-item (completed)="onTaskCompleted()"></app-todo-item>
```

Listing 4: p1-2-output-parent.component.html

Miniejercicio 1: Notificación de Cambio de Estado con @Output

Enunciado

Construye un componente hijo standalone denominado BotonContadorComponent que reciba un valor numérico inicial mediante @Input() y exponga un evento @Output() con EventEmitter<number> para notificar al padre cada vez que el valor aumente en una unidad.

Solución

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-boton-contador',
  standalone: true,
  template: `
    <button (click)="incrementar()" class="btn-step">
      Incrementar (Valor recibido: {{ paso }})
    </button>
  `,
  styles: [`
    .btn-step { background: #1E75B8; color: #fff; padding: 6px 12px; border: none; border-radius
      ↵ : 4px; }
  `]
})
```

```
})  
export class BotonContadorComponent {  
  @Input() paso: number = 1;  
  @Output() incremento = new EventEmitter<number>();  
  
  incrementar(): void {  
    this.incremento.emit(this.paso);  
  }  
}
```

Listing 5: p1-mini-boton-contador.component.ts

Explicación

- **Parametrización con @Input():** Permite al componente contenedor definir la magnitud del salto numérico sin acoplamiento interno.
- **Emisión tipada con EventEmitter:** Declaras `new EventEmitter<number>()`, garantizando que el dato emitido cumpla con el contrato de tipo y evitando inconsistencias en el componente receptor.

2 Servicios e Inyección de Dependencias (@Injectable)

La separación de responsabilidades dicta que la lógica de presentación debe residir en los componentes, mientras que las reglas de negocio, la persistencia y la comunicación con APIs externas deben ser delegadas a los Servicios.

2.1 El Decorador @Injectable

Para que una clase de TypeScript pueda actuar como un servicio consumible por otros componentes, debe estar decorada con @Injectable.

```
import { Injectable } from '@angular/core';  
  
@Injectable({  
  providedIn: 'root' // Define que el servicio es un Singleton a nivel global  
})  
export class DataService {  
  private data: string[] = [];  
  
  getItems(): string[] {  
    return this.data;  
  }  
  
  addItem(item: string): void {  
    this.data.push(item);  
  }  
}
```

Listing 6: p2-1-data.service.ts

El metadato `providedIn: 'root'` indica que Angular instanciará una única copia de este servicio (patrón Singleton) para toda la aplicación y lo registrará en el inyector raíz. Además, facilita el tree-shaking (ver ANEXO I), eliminando el servicio del bundle final si este no llega a

utilizarse.

Miniejercicio 2: Servicio de Auditoría Singleton con @Injectable

Enunciado

Define un servicio de registro de logs (AuditLogService) configurado como Singleton global mediante @Injectable(providedIn: 'root') que mantenga un historial inmutable de mensajes y provea métodos para agregar registros y consultar el total acumulado.

Solución

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuditLogService {
  private logs: string[] = [];

  registrarAccion(accion: string): void {
    const timestamp = new Date().toISOString();
    this.logs.push(`[${timestamp}] ${accion}`);
  }

  obtenerHistorial(): readonly string[] {
    return [...this.logs];
  }
}
```

Listing 7: p2-mini-audit-log.service.ts

Explicación

- **Singleton en inyector raíz:** Al declarar providedIn: 'root', compartes una única instancia del arreglo logs en toda la aplicación (ver ANEXO I).
- **Inmutabilidad defensiva:** El método obtenerHistorial() retorna una copia superficial ([...this.logs]) tipada como readonly, impidiendo alteraciones externas accidentales.

3 Ciclo de Vida de un Componente

Cada componente en Angular pasa por una serie de fases gestionadas directamente por el framework. Puedes interceptar estas fases implementando interfaces conocidas como Lifecycle Hooks (ver ANEXO II).

Hook	Método	Propósito y Momento de Ejecución
OnChanges	ngOnChanges()	Se ejecuta antes de ngOnInit y siempre que cambie un valor de un parámetro de entrada (@Input()).
OnInit	ngOnInit()	Inicialización del componente. Es el lugar idóneo para realizar llamadas a servicios para recuperar datos.
DoCheck	ngDoCheck()	Permite implementar un algoritmo de detección de cambios personalizado complementario al del framework.
AfterContentInit	ngAfterContentInit()	Se ejecuta después de que Angular proyecte contenido externo en la vista del componente (<ng-content>).
AfterViewInit	ngAfterViewInit()	Se ejecuta tras inicializar las vistas del componente y sus componentes hijos. Ideal para interactuar de forma segura con el DOM.
OnDestroy	ngOnDestroy()	Invocado inmediatamente antes de destruir el componente. Es imperativo usarlo para cancelar suscripciones activas o liberar recursos.

Cuadro 1: Ciclo de vida secuencial del componente

Miniejercicio 3: Control de Fugas de Memoria en ngOnDestroy

Enunciado

Implementa un componente standalone temporizador que inicie un intervalo de sincronización cada segundo en el gancho ngOnInit() y destruya limpiamente el temporizador en ngOnDestroy() para evitar pérdidas de memoria (memory leaks).

Solución

```
import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-timer-cleanup',
  standalone: true,
  template: `<p>Segundos transcurridos: {{ segundos }}</p>`
})
export class TimerCleanupComponent implements OnInit, OnDestroy {
  segundos: number = 0;
  private intervalId: any;

  ngOnInit(): void {
    this.intervalId = setInterval(() => {
      this.segundos++;
    }, 1000);
  }

  ngOnDestroy(): void {
    if (this.intervalId) {
      clearInterval(this.intervalId);
    }
  }
}
```

Listing 8: p3-mini-timer-cleanup.component.ts

Explicación

- **Inicialización segura:** `ngOnInit()` garantiza que el componente ya ha resuelto sus entradas antes de ejecutar tareas asíncronas.
- **Liberación mandatoria de recursos:** Si omites `clearInterval()` en `ngOnDestroy()`, el temporizador permanecerá en el event loop del navegador aun cuando el componente haya sido destruido del DOM.

4 Uso de `inject()` en lugar de Constructor para Inyección Moderna

A partir de Angular 14+, el framework ha introducido la función `inject()`, una alternativa funcional muy potente frente a la inyección tradicional basada en constructores.

4.1 Comparativa de Enfoques

Observa cómo cambia radicalmente la sintaxis de inyección y cómo mejora la legibilidad del código:

```
import { Component } from '@angular/core';
import { DataService } from '../data.service';

@Component({
  selector: 'app-old-way',
  standalone: true,
  template: `...`
})
export class OldWayComponent {
  constructor(private dataService: DataService) {
    // El servicio se inyecta y se asigna implícitamente a una propiedad
  }
}
```

Listing 9: p4-1-old-way.component.ts

```
import { Component, inject } from '@angular/core';
import { DataService } from '../data.service';

@Component({
  selector: 'app-modern-way',
  standalone: true,
  template: `...`
})
export class ModernWayComponent {
  private dataService = inject(DataService);
  // Propiedad tipada e inicializada directamente de forma limpia
}
```

Listing 10: p4-1-modern-way.component.ts

4.2 Ventajas Clave del Uso de inject()

- **Sin problemas de herencia:** En clases derivadas, ya no es necesario invocar la directiva `super()` pasándole todas las dependencias requeridas por el padre, solventando uno de los principales problemas del acoplamiento clásico.
- **Mejor inferencia de tipos:** El tipo de la variable es autodetectado con precisión, reduciendo código redundante.
- **Inyección fuera de clases:** Permite el desarrollo de funciones utilitarias y composiciones de lógica reutilizables (comúnmente conocidas como funciones composable o helpers funcionales), que pueden inyectar servicios fuera de la definición estricta de una clase de componente (ver ANEXO II).

Miniejercicio 4: Inyección Funcional en Composable Reutilizable

Enunciado

Crea una función utilitaria composable denominada `useAuditLog()` que utilice la función moderna `inject()` para obtener el servicio `AuditLogService` y exponga una función abreviada para registrar trazas en cliente.

Solución

```
import { inject } from '@angular/core';
import { AuditLogService } from './audit-log.service';

export function useAuditLog() {
  const auditService = inject(AuditLogService);

  return {
    registrar: (evento: string) => {
      auditService.registrarAccion(`[Operacion UI]: ${evento}`);
    }
  };
}
```

Listing 11: p4-mini-audit.composable.ts

Explicación

- **Inyección en funciones puras:** La función `inject()` permite resolver dependencias en el contexto de instanciación sin necesidad de heredar ni declarar constructores formales en clases.
- **Composabilidad modular:** Promueve una arquitectura desacoplada donde los componentes simplemente invocan `useAuditLog()` como atributo de clase.

5 Ejercicio Final

Lista de Tareas con Servicio Compartido

Enunciado

Construye una aplicación funcional de administración de tareas articulada en tres piezas de software desacopladas en Angular 19:

1. **TodoService:** Servicio singleton que gestiona el listado y estado de las tareas mediante operaciones estructuradas.
2. **TodoListComponent (Padre):** Componente standalone que inyecta el servicio mediante la función `inject()`, utiliza el control de flujo nativo del compilador (`@for`) y renderiza la colección.
3. **TodoItemComponent (Hijo):** Componente de presentación standalone que recibe la tarea mediante `@Input()` y notifica acciones con `@Output()`.

Solución

Paso 1: Creación del Servicio de Tareas

```
import { Injectable } from '@angular/core';

export interface Task {
  id: number;
  title: string;
  completed: boolean;
}

@Injectable({
  providedIn: 'root'
})
export class TodoService {
  private tasks: Task[] = [
    { id: 1, title: 'Configurar entorno Angular 19', completed: true },
    { id: 2, title: 'Dominar la inyeccion con inject()', completed: false }
  ];

  getTasks(): Task[] {
    return this.tasks;
  }

  addTask(title: string): void {
    const newTask: Task = {
      id: Date.now(),
      title,
      completed: false
    };
    this.tasks.push(newTask);
  }

  removeTask(id: number): void {
  }
```

```

    this.tasks = this.tasks.filter(task => task.id !== id);
  }

  toggleTask(id: number): void {
    const task = this.tasks.find(t => t.id === id);
    if (task) {
      task.completed = !task.completed;
    }
  }
}

```

Listing 12: p5-ex-todo.service.ts

Paso 2: Componente Hijo Parametrizable

```

import { Component, Input, Output, EventEmitter } from '@angular/core';
import { Task } from '../todo.service';

@Component({
  selector: 'app-todo-item',
  standalone: true,
  template: `
    <div class="todo-item" [style.text-decoration]="task.completed ? 'line-through' : 'none'">
      <span>{{ task.title }}</span>
      <button (click)="onToggle()">Cambiar Estado</button>
      <button (click)="onDelete()" style="color: red;">Eliminar</button>
    </div>
  `,
  styles: [`
    .todo-item {
      display: flex;
      justify-content: space-between;
      padding: 8px;
      border-bottom: 1px solid #ddd;
    }
  `]
})
export class TodoItemComponent {
  @Input() task!: Task;
  @Output() toggle = new EventEmitter<number>();
  @Output() delete = new EventEmitter<number>();

  onToggle(): void {
    this.toggle.emit(this.task.id);
  }

  onDelete(): void {
    this.delete.emit(this.task.id);
  }
}

```

Listing 13: p5-ex-todo-item.component.ts

Paso 3: Componente Padre Coordinador con Control de Flujo Moderno

```

import { Component, inject } from '@angular/core';
import { TodoService, Task } from '../todo.service';
import { TodoItemComponent } from '../todo-item.component';

```

```

@Component({
  selector: 'app-todo-list',
  standalone: true,
  imports: [TodoItemComponent],
  template: `
    <div class="todo-container">
      <h2>Mi Lista de Tareas</h2>
      <div class="input-group">
        <input #newTaskInput placeholder="Nueva tarea..." />
        <button (click)="addNewTask(newTaskInput.value); newTaskInput.value=''>
          Agregar Tarea
        </button>
      </div>
      <div class="list-wrapper">
        @for (t of todoService.getTasks(); track t.id) {
          <app-todo-item
            [task]="t"
            (toggle)="onToggleTask($event)"
            (delete)="onDeleteTask($event)">
          </app-todo-item>
        } @empty {
          <p class="empty-msg">No hay tareas pendientes en la lista.</p>
        }
      </div>
    </div>
  `,
  styles: [`
    .todo-container { max-width: 500px; margin: 0 auto; padding: 20px; }
    .input-group { margin-bottom: 15px; display: flex; gap: 10px; }
    .input-group input { flex-grow: 1; padding: 8px; }
    .input-group button { background-color: #2980b9; color: white; border: none; padding: 8px 12px; cursor: pointer; }
    .empty-msg { color: #64748B; font-style: italic; text-align: center; }
  `]
})
export class TodoListComponent {
  public todoService = inject(TodoService);

  addNewTask(title: string): void {
    if (title.trim()) {
      this.todoService.addTask(title);
    }
  }

  onToggleTask(id: number): void {
    this.todoService.toggleTask(id);
  }

  onDeleteTask(id: number): void {
    this.todoService.removeTask(id);
  }
}

```

Listing 14: p5-ex-todo-list.component.ts

Explicación

- **Control de Flujo Nativo del Compilador (@for):** Utilizas la sintaxis integrada de Angular 19 con seguimiento estricto por identificador (`track t.id`) y bloque declarativo `@empty`, descartando directivas de versiones obsoletas.
- **Desacoplamiento arquitectónico:** El componente `TodoItemComponent` actúa como un componente de presentación pura (*dumb component*), delegando la alteración real de los datos a través de eventos `EventEmitter` (ver ANEXO I).
- **Centralización del estado en servicio:** `TodoService` mantiene la única fuente de verdad (*single source of truth*), garantizando que las mutaciones sobre la colección sean consistentes y reutilizables.
- **Inyección funcional con `inject()`:** El componente `TodoListComponent` instancia el servicio sin constructor, simplificando el mantenimiento de dependencias y eliminando código redundante (ver ANEXO II).

6 Resumen Ejecutivo del Conocimiento Adquirido

La consolidación de los mecanismos de comunicación y servicios en Angular 19 sienta las bases de aplicaciones modulares, predecibles y de alto rendimiento:

- **Flujo Unidireccional Estricto:** La arquitectura jerárquica garantiza la coherencia del estado mediante el paso explícito de datos descendentes (`@Input()`) y la emisión ascendente de eventos tipados (`@Output()` y `EventEmitter`), favoreciendo la creación de componentes de presentación independientes y testeables (ver ANEXO I).
- **Centralización del Negocio con Inyección Jerárquica:** La separación entre vistas y lógica operativa a través de servicios decorados con `@Injectable({ providedIn: 'root' })` garantiza instancias Singleton seguras, eliminando la duplicidad de datos en memoria y maximizando el potencial del empaquetado con Tree-Shaking (ver ANEXO I).
- **Gobierno Preciso del Ciclo de Vida:** El dominio de los Lifecycle Hooks permite sincronizar con exactitud la inicialización de vistas (`ngOnInit`, `ngAfterViewInit`) y prevenir fugas de memoria críticas cancelando temporizadores o suscripciones en `ngOnDestroy` (ver ANEXO II).
- **Inyección Funcional Moderna con `inject()`:** La adopción de la función `inject()` optimiza la ergonomía del código TypeScript al simplificar la herencia de clases, posibilitar la inferencia precisa de tipos y habilitar la composición mediante helpers funcionales reutilizables fuera de clases tradicionales (ver ANEXO II).

ANEXO I: Glosario de Términos Técnicos

1. **Input Property (@Input):** Mecanismo mediante el cual un componente hijo expone una propiedad pública que puede ser vinculada por su componente contenedor (padre) mediante enlace de propiedad (*Property Binding*).
2. **Output Property (@Output):** Propiedad de un componente hijo asociada a una instancia de `EventEmitter` que despacha eventos hacia el componente padre en respuesta a interacciones o cambios de estado.
3. **Signal Inputs (input()):** Función reactiva introducida en Angular moderno que declara una entrada parametrizada como una señal de solo lectura (*Read-only Signal*).
4. **Model Inputs (model()):** Primitiva reactiva de doble vía que define una señal mutable compartida entre el componente padre y el hijo, sustituyendo el patrón clásico `[(ngModel)]`.
5. **EventEmitter:** Clase auxiliar derivada de RxJS utilizada en Angular para emitir eventos sincrónicos o asíncronos personalizados que transportan una carga útil (*payload*) tipada.
6. **Dependency Injection (DI):** Patrón de diseño de software en el cual un inyector provee instancias de servicios a los componentes o clases consumidoras sin que estas deban instanciarlas manualmente mediante el operador `new`.
7. **Hierarchical Injector:** Árbol estratificado de inyectores de dependencias en Angular donde un inyector hijo puede heredar o sobrescribir las instancias provistas por inyectores ancestros.
8. **Singleton:** Patrón creacional que garantiza que una clase solo tenga una única instancia activa en toda la aplicación, compartiendo el mismo estado en cualquier punto de consumo.
9. **Tree-Shaking:** Técnica de optimización en compilación que inspecciona las sentencias `import` y suprime del paquete final (*bundle*) cualquier servicio, clase o método no referenciado.
10. **Lifecycle Hooks:** Métodos de interfaz interceptores proporcionados por el núcleo de Angular que se ejecutan secuencialmente en fases concretas de la creación, renderizado o destrucción de un componente.
11. **Injection Context:** Momento específico de ejecución (por ejemplo, durante la inicialización de propiedades de clase o dentro de funciones de configuración) en el cual el inyector activo de Angular está disponible para invocar la función `inject()`.
12. **Dumb Component (Componente de Presentación):** Componente visual que carece de servicios de negocio o lógica compleja y cuya operativa se limita a recibir información por `@Input()` y emitir notificaciones por `@Output()`.
13. **Memory Leak (Fuga de Memoria):** Retención involuntaria de memoria producida cuando un componente es destruido del DOM pero mantiene tareas asíncronas, temporizadores o suscripciones vivas en el entorno de ejecución.
14. **takeUntilDestroyed:** Operador reactivo de RxJS provisto por Angular que cancela automáticamente una suscripción cuando el componente o contexto de inyección asociado es destruido.

ANEXO II: Matriz del Ciclo de Vida y Estrategias de Inyección

1 Matriz Operativa del Ciclo de Vida de Componentes

La siguiente matriz detalla el momento exacto de invocación, el propósito arquitectónico y los riesgos de implementación de cada hook en Angular 19:

Hook de Ciclo	Caso de Uso Recomendado	Riesgo Crítico a Evitar
ngOnChanges	Reaccionar a mutaciones en <code>@Input()</code> y calcular deltas.	Modificar variables locales sin control, generando bucles infinitos.
ngOnInit	Peticiones HTTP iniciales y suscripción a flujos de datos.	Acceder a referencias directas del DOM que aún no se han renderizado.
ngDoCheck	Detección de alteraciones profundas en objetos complejos.	Ejecutar algoritmos pesados de complejidad $\mathcal{O}(n^2)$ en cada microtarea.
ngAfterViewInit	Manipulación de vistas hijas mediante <code>@ViewChild</code> .	Mutar propiedades del componente provocando error <i>ExpressionChangedAfterChecked</i> .
ngOnDestroy	Cancelación de temporizadores, sockets y eventos globales.	Omitir la liberación de recursos reteniendo componentes destruidos en memoria.

Cuadro 2: Matriz de ingeniería de Lifecycle Hooks

Miniejercicio: Intercepción de Entradas con `ngOnChanges`

Enunciado: Desarrolla un componente que reciba un código de identificación por `@Input()` y registre en una lista interna el valor previo y el valor actual cada vez que la propiedad cambie mediante `ngOnChanges`.

Solución:

```
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';

@Component({
  selector: 'app-registro-cambios',
  standalone: true,
  template: `
    <p>ID actual: {{ idRegistro }}</p>
    <p>Ultimo cambio: {{ logCambio }}</p>
  `
})
export class RegistroCambiosComponent implements OnChanges {
  @Input() idRegistro: number = 0;
  logCambio: string = 'Sin variaciones';

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['idRegistro'] && !changes['idRegistro'].isFirstChange()) {
      const prev = changes['idRegistro'].previousValue;
      const curr = changes['idRegistro'].currentValue;
      this.logCambio = `Transicion: ${prev} -> ${curr}`;
    }
  }
}
```

Listing 15: anx2-p1-mini-changes.component.ts

Explicación: SimpleChanges permite acceder a previousValue y currentValue, permitiendo validar si se trata del primer ciclo con isFirstChange() y actuar en consecuencia.

2 Estrategias de Inyección: Constructor vs Función inject()

La inyección funcional con inject() democratiza el acceso a dependencias fuera de la estructura jerárquica de clases:

- **Inyección en Constructores:** Depende de la reflexión de metadatos de TypeScript (emitDecoratorMetadata). Obliga a clases hijas a transportar servicios requeridos por la clase base mediante super(servicioA, servicioB).
- **Inyección Funcional inject():** Opera en tiempo de inicialización de la instancia (*Injection Context*). Es invocable en funciones de factoría, funciones composables, inicializadores de campos y guards funcionales.

Miniejercicio: Helper Funcional de Enrutamiento con inject()

Enunciado: Construye una función composable reutilizable denominada useNavigationHelper() que inyecte Router y retorne un método directo para navegar al inicio de la aplicación.

Solución:

```
import { inject } from '@angular/core';
import { Router } from '@angular/router';

export function useNavigationHelper() {
  const router = inject(Router);

  return {
    irAlInicio: () => router.navigate([''])
  };
}
```

Listing 16: anx2-p2-mini-inject-helper.ts

Explicación: Al invocar inject(Router) dentro de la función, esta puede ser consumida directamente como inicializador de propiedad en cualquier componente autónomo.

3 Ejercicio final Matriz del Ciclo de Vida y Estrategias de Inyección

Consola de Diagnóstico de Componente con Auditoría Integral de Ciclo de Vida

Enunciado

Construye un componente de monitoreo técnico standalone denominado ConsolaDiagnosticoComponent que intercepte secuencialmente ngOnInit,

ngAfterViewInit y ngOnDestroy. El componente debe inyectar un servicio de telemetría mediante inject() y almacenar marcas de tiempo en una lista reactiva, garantizando la destrucción segura de sus subprocesos.

Solución

```
import { Component, OnInit, AfterViewInit, OnDestroy, inject } from '@angular/core';
import { AuditLogService } from './audit-log.service';

@Component({
  selector: 'app-consola-diagnostico',
  standalone: true,
  template: `
    <div class="consola-card">
      <h4>Eventos Registrados de Ciclo de Vida</h4>
      <ul>
        @for (evento of historial; track evento) {
          <li>{{ evento }}</li>
        }
      </ul>
    </div>
  `,
  styles: [`
    .consola-card { background: #F8FBFE; border: 1px solid #1E75B8; padding: 15px; border-radius
      ↪ : 6px; }
  `]
})
export class ConsolaDiagnosticoComponent implements OnInit, AfterViewInit, OnDestroy {
  private audit = inject(AuditLogService);
  historial: string[] = [];

  ngOnInit(): void {
    this.registrar('Fase 1: ngOnInit ejecutado exitosamente');
  }

  ngAfterViewInit(): void {
    this.registrar('Fase 2: ngAfterViewInit completado (Vistas renderizadas)');
  }

  ngOnDestroy(): void {
    this.audit.registrarAccion('Fase 3: ngOnDestroy invocado (Liberacion completada)');
  }

  private registrar(mensaje: string): void {
    this.historial.push(`${new Date().toLocaleTimeString()} ${mensaje}`);
    this.audit.registrarAccion(mensaje);
  }
}
```

Listing 17: anx2-final-diagnostico-ciclo.component.ts

Explicación

- **Resolución Funcional de Dependencias:** Utilizas inject(AuditLogService) como atributo de clase, desterrando el constructor redundante.

- **Garantía de Secuencia:** Se audita la transición desde la inicialización del modelo (`ngOnInit`) hasta la pintura final del DOM (`ngAfterViewInit`) y la clausura limpia de recursos (`ngOnDestroy`).

ANEXO III: Nuevas Primitivas de Señales en Comunicación

Angular 19 consolida la transición hacia primitivas reactivas basadas en funciones, simplificando la sincronización de datos y superando los decoradores clásicos.

1 Signal Inputs: `input()` e `input.required()`

La función `input()` declara una entrada del componente como una señal de solo lectura (*Signal*). Elimina la necesidad del decorador `@Input()` y se evalúa de manera reactiva:

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-etiqueta-alerta',
  standalone: true,
  template: `<span>{{ mensaje() }} - Criticidad: {{ nivel() }}</span>`
})
export class EtiquetaAlertaComponent {
  // Entrada opcional con valor por defecto
  mensaje = input<string>('Sin alertas');
  // Entrada obligatoria validada por el compilador
  nivel = input.required<number>();
}
```

Listing 18: anx3-1-signal-input-demo.ts

Miniejercicio: Transformación Reactiva con `input()`

Enunciado: Desarrolla un componente standalone que reciba un precio numérico mediante `input.required<number>()` y exponga una señal computada (`computed()`) que calcule automáticamente el importe con el 21% de IVA.

Solución:

```
import { Component, input, computed } from '@angular/core';

@Component({
  selector: 'app-precio-iva',
  standalone: true,
  template: `<p>Base: {{ base() }} EUR | Con IVA: {{ totalConIva() }} EUR</p>`
})
export class PrecioIvaComponent {
  base = input.required<number>();
  totalConIva = computed(() => (this.base() * 1.21).toFixed(2));
}
```

Listing 19: anx3-p1-mini-signal-input.component.ts

Explicación: La señal computada `totalConIva` rastrea de forma atómica el valor de `base()`, recalculando el total sin requerir ganchos `ngOnChanges`.

2 Signal Outputs: `output()`

La función `output()` sustituye al decorador clásico `@Output()` y a `EventEmitter`, proporcionando una interfaz de emisión limpia y con menor sobrecarga de memoria:

```
import { Component, output } from '@angular/core';
```

```

@Component({
  selector: 'app-accion-rapida',
  standalone: true,
  template: `<button (click)="ejecutar()">Lanzar Accion</button>`
})
export class AccionRapidaComponent {
  onAccion = output<string>();

  ejecutar(): void {
    this.onAccion.emit('Operacion completada');
  }
}

```

Listing 20: anx3-2-signal-output-demo.ts

Miniejercicio: Emisión Declarativa con output()

Enunciado: Diseña un selector de color que emita una cadena hexadecimal hacia su componente contenedor cada vez que el usuario seleccione una opción mediante la función output().

Solución:

```

import { Component, output } from '@angular/core';

@Component({
  selector: 'app-selector-color',
  standalone: true,
  template: `
    <button (click)="elegir('#1E75B8')">Azul</button>
    <button (click)="elegir('#059669')">Verde</button>
  `
})
export class SelectorColorComponent {
  colorSeleccionado = output<string>();

  elegir(color: string): void {
    this.colorSeleccionado.emit(color);
  }
}

```

Listing 21: anx3-p2-mini-signal-output.component.ts

Explicación: La primitiva output() crea un emisor fuertemente tipado que el componente padre enlaza mediante enlace de eventos estándar (colorSeleccionado)="onCambioColor(\$event)".

3 Enlace Bidireccional Moderno: model()

La primitiva model() crea una señal de doble vía compartida entre el componente hijo y el padre. Si el componente hijo actualiza el modelo con .set() o .update(), el nuevo valor se propaga automáticamente hacia la propiedad vinculada en el componente padre:

```

import { Component, model } from '@angular/core';

@Component({
  selector: 'app-interruptor',
  standalone: true,

```

```

template: `
  <button (click)="toggle()">
    Estado: {{ activo() ? 'ENCENDIDO' : 'APAGADO' }}
  </button>
`
})
export class InterruptorComponent {
  activo = model<boolean>(false);

  toggle(): void {
    this.activo.update(estado => !estado);
  }
}

```

Listing 22: anx3-3-signal-model-demo.ts

Miniejercicio: Sincronización Bidireccional con model()

Enunciado: Implementa un control de volumen infantil con `model<number>(50)` que permita decrementar e incrementar el valor numérico en pasos de 5 unidades, actualizando automáticamente el componente contenedor.

Solución:

```

import { Component, model } from '@angular/core';

@Component({
  selector: 'app-control-volumen',
  standalone: true,
  template: `
    <button (click)="bajar()">-</button>
    <span>Volumen: {{ volumen() }}%</span>
    <button (click)="subir()">+</button>
  `
})
export class ControlVolumenComponent {
  volumen = model<number>(50);

  subir(): void {
    this.volumen.update(v => Math.min(100, v + 5));
  }

  bajar(): void {
    this.volumen.update(v => Math.max(0, v - 5));
  }
}

```

Listing 23: anx3-p3-mini-model-counter.component.ts

Explicación: El componente padre puede consumir este hijo mediante sintaxis de doble vía `[(volumen)]="volumenPadre"`, sincronizando el estado sin declarar eventos `@Output()` manuales.

4 Ejercicio final Nuevas Primitivas de Señales en Comunicación

Editor Parametrizable Reactivo con Primitivas de Señales en Angular 19

Enunciado

Construye un editor de configuración que coordine un componente padre (ConfiguradorPadreComponent) y un componente hijo (EditorTarjetaComponent) utilizando exclusivamente las nuevas primitivas de señales en Angular 19:

1. **input.required<string>():** Para recibir el identificador inmutable de la tarjeta.
2. **model<string>():** Para el título editable de la tarjeta sincronizado en tiempo real.
3. **output<void>():** Para emitir el evento de guardado definitivo al padre.

Solución

```
import { Component, input, model, output } from '@angular/core';

@Component({
  selector: 'app-editor-tarjeta',
  standalone: true,
  template: `
    <div class="tarjeta-box">
      <small>ID: {{ tarjetaId() }}</small>
      <input
        [value]="titulo()"
        (input)="actualizarTitulo($event)"
        placeholder="Titulo de la tarjeta" />
      <button (click)="guardarCambios()">Confirmar</button>
    </div>
  `,
  styles: [`
    .tarjeta-box { border: 1px solid #CBD5E1; padding: 12px; border-radius: 6px; background: #
      ↪ fff; }
    input { margin: 0 8px; padding: 4px; }
  `]
})
export class EditorTarjetaComponent {
  tarjetaId = input.required<string>();
  titulo = model<string>('');
  guardado = output<void>();

  actualizarTitulo(event: Event): void {
    const input = event.target as HTMLInputElement;
    this.titulo.set(input.value);
  }

  guardarCambios(): void {
    this.guardado.emit();
  }
}
```

Listing 24: anx3-final-editor-reactivo.component.ts

Explicación

- **Inmutabilidad y Reactividad Fina:** `tarjetaId` no puede ser modificado por el hijo al ser una señal de solo lectura provista por `input.required()`.
- **Eliminación de Boilerplate:** La primitiva `model()` reemplaza de forma simultánea un `@Input()` y un `@Output()` `tituloChange`, simplificando la arquitectura a un único punto de enlace reactivo.

ANEXO IV: Ámbitos de Inyección y Jerarquía de Inyectores

El inyector jerárquico de Angular permite configurar el ciclo de vida y la visibilidad de los servicios en distintos niveles del árbol de componentes.

1 Inyector Raíz (providedIn: 'root')

Un servicio configurado con `providedIn: 'root'` se registra en el *Root Environment Injector*.

- Es un **Singleton global**: todos los componentes consumen exactamente la misma instancia.
- Facilita la eliminación de código muerto (*Tree-Shaking*) si no es consumido.

Miniejercicio: Servicio de Notificaciones Global

Enunciado: Implementa un servicio singleton registrado en la raíz denominado `NotificacionGlobalService` que exponga un método para despachar mensajes en consola accesibles desde cualquier componente.

Solución:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class NotificacionGlobalService {
  emitirAlerta(msg: string): void {
    console.info(`[ALERTA GLOBAL]: ${msg}`);
  }
}
```

Listing 25: anx4-p1-mini-root-service.ts

Explicación: Al usar `providedIn: 'root'`, el servicio es instanciado bajo demanda y perdura durante toda la sesión activa de la aplicación.

2 Proveedores Locales de Componente (providers: [...])

Cuando un componente declara un servicio en su arreglo `providers: [...]`, Angular crea una instancia aislada para dicho componente y todos sus hijos:

```
@Component({
  selector: 'app-formulario-pedido',
  standalone: true,
  providers: [PedidoStateService] // Instancia unica para este subarbol
})
export class FormularioPedidoComponent {}
```

Listing 26: anx4-2-scoped-provider.component.ts

Miniejercicio: Estado Aislado por Componente

Enunciado: Diseña un componente `FichaClienteComponent` que declare un servicio de estado local en su metadato `providers` para asegurar que cada ficha mantenga sus pro-

pios borradores independientes.

Solución:

```
import { Component, Injectable, inject } from '@angular/core';

@Injectable()
export class BorradorService {
  public textoBorrador = '';
}

@Component({
  selector: 'app-ficha-cliente',
  standalone: true,
  providers: [BorradorService],
  template: `<p>Borrador activo: {{ borrador.textoBorrador }}</p>`
})
export class FichaClienteComponent {
  public borrador = inject(BorradorService);
}
```

Listing 27: anx4-p2-mini-component-service.ts

Explicación: Cada vez que Angular crea un `<app-ficha-cliente>`, instancia un `BorradorService` dedicado, destruyéndolo cuando el componente sale del DOM.

3 Modificadores de Resolución de Dependencias

En Angular moderno con `inject()`, puedes configurar reglas de resolución mediante opciones:

- **optional:** Devuelve `null` si el servicio no está provisto en ningún nivel: `inject(Token, { optional: true })`.
- **self:** Restringe la búsqueda de la dependencia exclusivamente al inyector del propio componente: `inject(Token, { self: true })`.

Miniejercicio: Inyección Opcional Segura

Enunciado: Desarrolla un componente que intente inyectar un servicio de telemetría opcional (`AnalyticsService`) utilizando `inject()` y valide su existencia antes de registrar acciones.

Solución:

```
import { Component, inject } from '@angular/core';

export abstract class AnalyticsService {
  abstract enviar(tag: string): void;
}

@Component({
  selector: 'app-boton-rastreador',
  standalone: true,
  template: `<button (click)="clickRastreado()">Pulsar Accion</button>`
})
```

```
export class BotonRastreadorComponent {
  private analytics = inject(AnalyticsService, { optional: true });

  clickRastreado(): void {
    if (this.analytics) {
      this.analytics.enviar('btn_click');
    }
  }
}
```

Listing 28: anx4-p3-mini-optional-self.component.ts

Explicación: La opción { optional: true } previene el error *NullInjectorError* si ningún proveedor ha registrado la clase en el árbol.

4 Ejercicio final Ámbitos de Inyección y Jerarquía de Inyectores

Sistema de Pestañas con Contadores Independientes mediante Proveedores Locales

Enunciado

Construye un sistema de pestañas independientes donde cada pestaña instancie su propio servicio de contador (*PestanaCounterService*) declarado a nivel de componente con providers: [...]. La vista principal debe montar dos pestañas contiguas y demostrar que incrementar el contador en una no altera el contador de la otra.

Solución

```
import { Component, Injectable, inject } from '@angular/core';

@Injectable()
export class PestanaCounterService {
  contador: number = 0;
  incrementar(): void { this.contador++; }
}

@Component({
  selector: 'app-pestana-individual',
  standalone: true,
  providers: [PestanaCounterService],
  template: `
    <div class="pestana">
      <p>Valor de esta pestana: {{ srv.contador }}</p>
      <button (click)="srv.incrementar()">Incrementar</button>
    </div>
  `,
  styles: [`.pestana { border: 1px solid #1E75B8; padding: 10px; margin: 5px; }`]
})
export class PestanaIndividualComponent {
  public srv = inject(PestanaCounterService);
}
```

Listing 29: anx4-final-contador-aislado.component.ts

Explicación

- **Aislamiento de Instancias:** Al registrar providers: [PestanaCounterService] dentro del decorador del componente, cada instancia visual posee un inyector local independiente.
- **Ciclo de Vida Acoplado:** Cuando el usuario cierra o destruye una pestaña, Angular destruye automáticamente la instancia asociada del servicio, liberando la memoria consumida.

ANEXO V: Prevención de Fugas de Memoria en Componentes

Las fugas de memoria (*Memory Leaks*) suceden cuando un componente es removido del DOM pero mantiene suscripciones activas a flujos infinitos o manejadores de eventos.

1 Identificación de Fugas por Suscripciones Infinitas

Las llamadas HTTP mediante `HttpClient` se completan automáticamente tras emitir un valor. Sin embargo, flujos derivados de `interval`, `fromEvent` o un `BehaviorSubject` permanecen activos indefinidamente a menos que sean cancelados explícitamente.

Miniejercicio: Detección de Fuga de Memoria

Enunciado: Analiza el siguiente fragmento e identifica por qué retiene memoria una vez destruido el componente, corrigiendo la fuga mediante cancelación manual en `ngOnDestroy`.

Solución:

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { interval, Subscription } from 'rxjs';

@Component({
  selector: 'app-radar-fuga',
  standalone: true,
  template: `<p>Telemetria activa</p>`
})
export class RadarFugaComponent implements OnInit, OnDestroy {
  private sub!: Subscription;

  ngOnInit(): void {
    this.sub = interval(1000).subscribe(val => console.log(val));
  }

  ngOnDestroy(): void {
    if (this.sub) {
      this.sub.unsubscribe(); // Liberacion mandatoria
    }
  }
}
```

Listing 30: anx5-p1-mini-memory-leak-demo.component.ts

Explicación: Al almacenar la instancia de `Subscription` e invocar `.unsubscribe()` en `ngOnDestroy()`, se cancela la tarea del emisor en el temporizador del navegador.

2 El Operador Moderno `takeUntilDestroyed`

En Angular 16+ y consolidado en Angular 19, el operador `takeUntilDestroyed()` de `@angular/core/rxjs-interop` cancela automáticamente una suscripción cuando el componente se destruye, sin necesidad de declarar ganchos de ciclo de vida:

```
import { Component, inject } from '@angular/core';
import { interval } from 'rxjs';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-reloj-seguro',
```

```

    standalone: true,
    template: `<p>Reloj funcionando de forma segura</p>`
  })
  export class RelojSeguroComponent {
    constructor() {
      interval(1000)
        .pipe(takeUntilDestroyed()) // Detecta el Injection Context actual
        .subscribe(tick => console.log('Tick:', tick));
    }
  }
}

```

Listing 31: anx5-2-take-until-demo.ts

Miniejercicio: Desuscripción Reactiva con takeUntilDestroyed

Enunciado: Desarrolla un componente standalone que escuche los cambios de tamaño de ventana con `fromEvent(window, 'resize')` y gestione la desconexión segura mediante `takeUntilDestroyed()`.

Solución:

```

import { Component, DestroyRef, inject, OnInit } from '@angular/core';
import { fromEvent } from 'rxjs';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-monitor-resize',
  standalone: true,
  template: `<p>Monitor de dimension de ventana en ejecucion</p>`
})
export class MonitorResizeComponent implements OnInit {
  private destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    fromEvent(window, 'resize')
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(() => console.log('Resize detectado'));
  }
}

```

Listing 32: anx5-p2-mini-take-until-destroyed.component.ts

Explicación: Cuando la suscripción se inicia fuera del constructor (por ejemplo en `ngOnInit`), se suministra la referencia `DestroyRef` inyectada al operador para ligar su destrucción.

3 Consumo Declarativo con AsyncPipe

El pipe `async` se suscribe automáticamente a un `Observable` desde el HTML y gestiona la cancelación de forma desatendida cuando la vista desaparece:

```

<!-- El pipe async se desuscribe sin codigo imperativo en TypeScript -->
<p>Precio de cotizacion: {{ cotizacion$ | async }}</p>

```

Listing 33: anx5-3-async-pipe-demo.html

Miniejercicio: Consumo Declarativo con AsyncPipe

Enunciado: Desarrolla un componente standalone que exponga un flujo observable `horaActual$` emitido cada segundo y lo renderice directamente en la plantilla con `AsyncPipe` sin suscripciones manuales.

Solución:

```
import { Component } from '@angular/core';
import { AsyncPipe } from '@angular/common';
import { interval, map, Observable } from 'rxjs';

@Component({
  selector: 'app-reloj-declarativo',
  standalone: true,
  imports: [AsyncPipe],
  template: `<p>Hora del sistema: {{ horaActual$ | async }}</p>`
})
export class RelojDeclarativoComponent {
  horaActual$: Observable<string> = interval(1000).pipe(
    map(() => new Date().toLocaleTimeString())
  );
}
```

Listing 34: `anx5-p3-mini-async-pipe.component.ts`

Explicación: Al utilizar `AsyncPipe`, Angular delega la apertura y cierre de la suscripción al ciclo de vida del nodo en el DOM, garantizando inmunidad frente a fugas de memoria.

4 Ejercicio final Prevención de Fugas de Memoria en Componentes

Panel de Telemetría Blindado contra Pérdidas de Memoria

Enunciado

Construye un componente standalone de visualización de telemetría en tiempo real (`MonitorTelemetriaComponent`) que consuma un flujo de datos continuo (`interval(500)`). La solución debe blindarse contra fugas de memoria utilizando `takeUntilDestroyed` y mantener un indicador visual del total de paquetes procesados.

Solución

```
import { Component, inject } from '@angular/core';
import { interval } from 'rxjs';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-monitor-telemetria',
  standalone: true,
  template: `
    <div class="telemetria-box">
      <h3>Monitor de Telemetría en Vivo</h3>
      <p>Paquetes recibidos: <strong>{{ totalPaquetes }}</strong></p>
    </div>
  `
})
export class MonitorTelemetriaComponent {
  totalPaquetes = 0;

  ngOnInit() {
    interval(500).pipe(
      takeUntilDestroyed()
    ).subscribe(() => {
      this.totalPaquetes++;
    });
  }
}
```

```
    </div>
  `,
  styles: [`
    .telemetria-box { border: 2px solid #059669; background: #F0FDF4; padding: 15px; border-
    ↪ radius: 8px; }
  `]
})
export class MonitorTelemetriaComponent {
  totalPaquetes: number = 0;

  constructor() {
    interval(500)
      .pipe(takeUntilDestroyed())
      .subscribe(() => {
        this.totalPaquetes++;
      });
  }
}
```

Listing 35: anx5-final-telemetria-blindada.component.ts

Explicación

- **Destrucción Contextual Automática:** Al situar el operador `takeUntilDestroyed()` dentro del constructor, Angular deduce automáticamente el *Injection Context*, finalizando la suscripción de forma limpia al destruirse el componente.
- **Cero Boilerplate:** Se erradica la necesidad de implementar `ngOnDestroy`, declarar variables `Subscription` o gestionar arreglos de desuscripción imperativos.

ANEXO VI: Fundamentos de TypeScript para Servicios

Un diseño de servicios profesional se apoya en el sistema de tipos estático de TypeScript para blindar la lógica de negocio frente a inconsistencias de datos en tiempo de desarrollo.

1 Modelado con Interfaces y Tipos de Dominio

Toda entidad de negocio procesada por un servicio debe estar tipada mediante interfaces explícitas:

```
export interface Usuario {  
  readonly id: number;  
  nombre: string;  
  email: string;  
  rol: 'ADMIN' | 'USUARIO';  
}
```

Listing 36: anx6-1-interfaces-dominio.ts

Miniejercicio: Modelado Inmutable de Entidad

Enunciado: Modela una interfaz TypeScript denominada `TransaccionBancaria` que cuente con identificador alfanumérico inmutable (`readonly`), importe numérico y fecha de emisión.

Solución:

```
export interface TransaccionBancaria {  
  readonly uuid: string;  
  monto: number;  
  fecha: Date;  
  tipo: 'INGRESO' | 'GASTO';  
}
```

Listing 37: anx6-p1-mini-interfaces-inmutables.ts

Explicación: El modificador `readonly` instruye al compilador a arrojar un error estático si algún método del servicio intenta reasignar la clave primaria.

2 Encapsulación y Modificadores de Acceso

Los estados internos del servicio deben declararse como `private`, ofreciendo acceso de solo lectura mediante métodos públicos o señales:

```
import { Injectable, signal } from '@angular/core';  
  
@Injectable({ providedIn: 'root' })  
export class SessionService {  
  private _usuarioActivo = signal<string | null>(null);  
  // Exposición de solo lectura para los componentes  
  public readonly usuarioActivo = this._usuarioActivo.asReadonly();  
  
  iniciarSesion(nombre: string): void {  
    this._usuarioActivo.set(nombre);  
  }  
}
```

Listing 38: anx6-2-encapsulacion-state.ts

Miniejercicio: Protección Defensiva de Estado

Enunciado: Desarrolla un servicio de carrito de compras que encapsule una colección privada de cadenas de texto y devuelva una copia de lectura en su método consultor para impedir modificaciones directas.

Solución:

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class CarritoService {
  private articulos: string[] = [];

  agregar(articulo: string): void {
    this.articulos.push(articulo);
  }

  obtenerArticulos(): readonly string[] {
    return [...this.articulos];
  }
}
```

Listing 39: anx6-p2-mini-encapsulacion-privada.ts

Explicación: Retornar `[...this.articulos]` genera una copia superficial (*shallow copy*), preservando la integridad del estado interno del servicio.

3 Servicios Genéricos Parametrizados <T>

Los tipos genéricos permiten diseñar servicios abstractos reutilizables (como clientes de almacenamiento o repositorios CRUD):

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class MemoryStoreService<T> {
  private elementos: T[] = [];

  almacenar(item: T): void {
    this.elementos.push(item);
  }

  listar(): readonly T[] {
    return [...this.elementos];
  }
}
```

Listing 40: anx6-3-servicio-generico.ts

Miniejercicio: Servicio Genérico de Pila (Stack)

Enunciado: Implementa un servicio genérico `PilaOperacionesService<T>` que provea los métodos `apilar(elemento: T)` y `desapilar(): T | undefined`.

Solución:

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class PilaOperacionesService<T> {
  private pila: T[] = [];

  apilar(elemento: T): void {
    this.pila.push(elemento);
  }

  desapilar(): T | undefined {
    return this.pila.pop();
  }
}
```

Listing 41: anx6-p3-mini-servicio-generico.ts

Explicación: El parámetro genérico <T> dota al servicio de versatilidad total, permitiendo apilar cadenas, objetos de entidades o números con estricta validación estática de tipos.

4 Ejercicio final Fundamentos de TypeScript para Servicios

Repositorio en Memoria Tipado y Genérico para Entidades de Negocio

Enunciado

Diseña un servicio genérico parametrizado denominado `EntidadRepositorioService<T extends { id: number }>` configurado como `providedIn: 'root'` que implemente operaciones de inserción, búsqueda por identificador y eliminación inmutable sobre cualquier entidad tipada de la aplicación.

Solución

```
import { Injectable } from '@angular/core';

export interface Identificable {
  id: number;
}

@Injectable({
  providedIn: 'root'
})
export class EntidadRepositorioService<T extends Identificable> {
  private datos: T[] = [];

  insertar(entidad: T): void {
    this.datos.push(entidad);
  }

  buscarPorId(id: number): T | undefined {
    return this.datos.find(e => e.id === id);
  }
}
```

```
}

eliminarPorId(id: number): void {
  this.datos = this.datos.filter(e => e.id !== id);
}

obtenerColeccion(): readonly T[] {
  return [...this.datos];
}
}
```

Listing 42: anx6-final-repositorio-tipado.service.ts

Explicación

- **Restricción Genérica** (`extends Identificable`): Garantiza en tiempo de compilación que cualquier clase o interfaz gestionada por el repositorio contenga un atributo numérico `id`.
- **Polimorfismo Seguro**: Un único servicio puede ser utilizado para gestionar usuarios, tareas o productos sin duplicar lógica y preservando la asistencia del autocompletado en el editor.